

第 6 章：查询执行

2026 版

邹兆年

哈尔滨工业大学
计算学部
海量数据计算研究中心
电子邮件: znzou@hit.edu.cn

2026 春

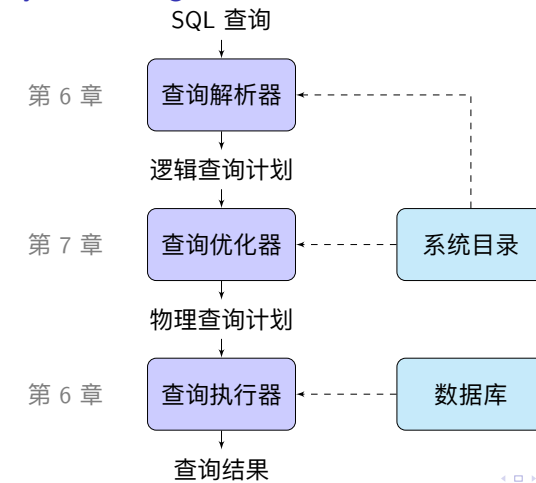
教学内容¹

- ① 6.1 查询处理的过程
- ② 6.2 物理查询算子的实现算法
 - 6.2.1 排序算子
 - 6.2.2 选择算子
 - 6.2.3 投影算子
 - 6.2.4 去重算子
 - 6.2.5 聚集算子
 - 6.2.6 集合算子
 - 6.2.7 连接算子
- ③ 6.3 查询执行模型
 - 6.3.1 物化执行
 - 6.3.2 流水线执行
 - 6.3.3 火山模型/迭代器模型
 - 6.3.4 查询执行的优化

¹课件更新于 2026 年 5 月 18 日

6.1 查询处理的过程

查询处理 (Query Processing) 的基本过程



查询解析器 (Query Parser)

将 SQL 查询转换成逻辑查询计划 (Logical Query Plan)

- 逻辑查询计划由逻辑查询操作构成, 接近关系代数表达式

例 (查询解析)

- 关系: $Student(Sno, Sname, Ssex, Sage, Sdept)$
- SQL 查询: `SELECT Sno, Sage FROM Student WHERE Sage > 18;`
- 逻辑查询计划: $\Pi_{Sno, Sage}(\sigma_{Sage > 18}(Student))$

查询解析的过程

- 1 词法分析
- 2 语法分析
- 3 语义分析
- 4 逻辑查询计划生成

词法分析

- 从左至右扫描 SQL 语句, 产生 Token 序列
- 词法分析器 lex

例 (词法分析)

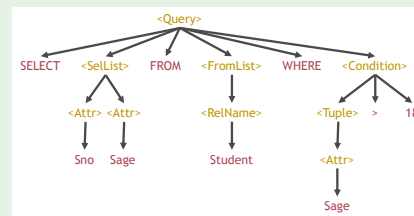
SELECT Sno , Sage FROM Student WHERE Sage > 18 ;

语法分析

- 将 Token 序列转成抽象语法树 (Abstract Syntax Tree, AST), 表示 SQL 语句的语法结构
- 语法分析器 yacc

例 (语法分析)

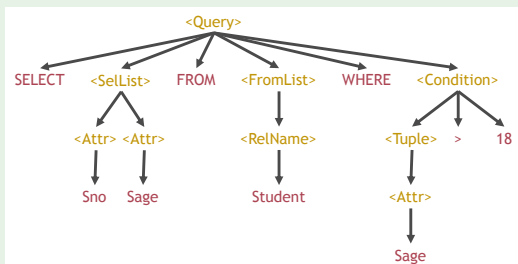
SELECT Sno, Sage FROM Student WHERE Sage > 18;



语义分析

- 检查关系和属性是否存在
- 检查数据类型
- 检查访问权限
- 生成逻辑查询计划

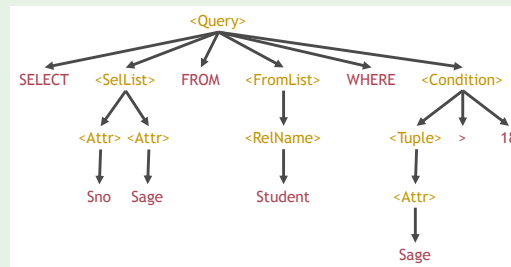
例 (语义分析)



逻辑查询计划生成

- 逻辑查询计划由逻辑查询算子 (Logical Query Operator) 构成
- 可以表示为逻辑查询计划树 (Logical Query Plan Tree)

例 (逻辑查询计划生成)



$\Pi_{Sno, Sage}$
|
 $\sigma_{Sage > 18}$
|
Student

查询优化器 (Query Optimizer)

将逻辑查询计划转换为执行效率“最高”的物理查询计划 (Physical Query Plan)

例 (查询优化)

初始逻辑查询计划

$\Pi_{Sno, Sage}$
|
 $\sigma_{Sage > 18}$
|
Student

等价的逻辑查询计划

$\sigma_{Sage > 18}$
|
 $\Pi_{Sno, Sage}$
|
Student

最优物理查询计划

$\Pi_{Sno, Sage}$
|
 $\sigma_{Sage > 18}$; 使用索引
|
Student

物理查询执行计划 (Physical Query Execution Plan)

- 物理查询算子 (Physical Query Operator) 是注释了“如何物理执行”的查询算子
- 物理查询计划由物理查询算子构成

例 (物理查询执行计划)

逻辑查询计划

$\Pi_{Sno, Sage}$
|
 $\sigma_{Sage > 18}$
|
Student

物理查询计划

$\Pi_{Sno, Sage}$
|
 $\sigma_{Sage > 18}$; 使用索引
|
Student

查询执行器 (Query Executior)

按照物理查询计划执行查询

- 物理查询算子的执行
- 物理算子间的数据传递

例 (查询执行)



6.2 物理查询算子的实现算法

排序

排序是 RDBMS 中非常重要的操作

- 使用 ORDER BY 对查询结果排序 (第 2 章)
- 批量加载 B+ 树的第一步是对索引项排序 (第 5 章)
- 排序是物理查询算子实现算法的重要步骤 (第 6 章)

6.2.1 排序算子

两趟多路外存归并排序 (Two-Pass Multi-Way External Merge Sort)

- 阶段 1: 创建归并段 (Run)
- 阶段 2: 归并 (Merge)

例 (两趟多路外存归并排序)

将关系中的元组按 id 属性排序



记法

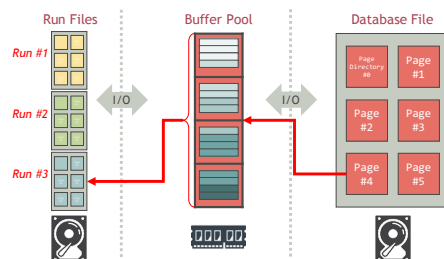
- $T(R)$: R 的元组数
- $B(R)$: R 的页数
- M : 缓冲池可用内存页数

阶段 1: 创建归并段

将关系 R 划分为 $\lceil B(R)/M \rceil$ 个归并段

过程: 创建归并段

- for R 文件的每 M 页 do
- 将这 M 个文件页读入缓冲池中可用的 M 页
- 对缓冲池中这 M 个页的元组按排序键排序, 形成归并段
- 将排好序的归并段写入一个新的归并段文件



阶段 1: 创建归并段

例 (创建归并段)

- $R = [2\ 5\ 2\ 1\ 2\ 2\ 4\ 5\ 4\ 3\ 4\ 2\ 1\ 5\ 2\ 1\ 3]$ (每个数代表一个元组)
- $N = 17$ (17 个元组)
- $M = 3$ (3 个可用内存页)
- $B(R) = 9$ (R 包含 9 个页)

创建 3 个归并段

- $2\ 5\ 2\ 1\ 2\ 2 \rightarrow R_1 = [1\ 2\ 2\ 2\ 2\ 5]$
- $4\ 5\ 4\ 3\ 4\ 2 \rightarrow R_2 = [2\ 3\ 4\ 4\ 4\ 5]$
- $1\ 5\ 2\ 1\ 3 \rightarrow R_3 = [1\ 1\ 2\ 3\ 5]$

阶段 1: 创建归并段

例 (创建归并段)

- $R = [2\ 5\ 2\ 1\ 2\ 2\ 4\ 5\ 4\ 3\ 4\ 2\ 1\ 5\ 2\ 1\ 3]$ (每个数代表一个元组)
- $N = 17$ (17 个元组)
- $M = 3$ (3 个可用内存页)
- $B(R) = 9$ (R 包含 9 个页)

创建 3 个归并段

- 1 $[2\ 5\ 2\ 1\ 2\ 2] \rightarrow R_1 = [1\ 2\ 2\ 2\ 2\ 5]$
- 2 $[4\ 5\ 4\ 3\ 4\ 2] \rightarrow R_2 = [2\ 3\ 4\ 4\ 4\ 5]$
- 3 $[1\ 5\ 2\ 1\ 3] \rightarrow R_3 = [1\ 1\ 2\ 3\ 5]$

阶段 1: 创建归并段

例 (创建归并段)

- $R = [2\ 5\ 2\ 1\ 2\ 2\ 4\ 5\ 4\ 3\ 4\ 2\ 1\ 5\ 2\ 1\ 3]$ (每个数代表一个元组)
- $N = 17$ (17 个元组)
- $M = 3$ (3 个可用内存页)
- $B(R) = 9$ (R 包含 9 个页)

创建 3 个归并段

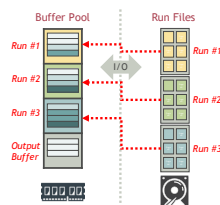
- 1 $[2\ 5\ 2\ 1\ 2\ 2] \rightarrow R_1 = [1\ 2\ 2\ 2\ 2\ 5]$
- 2 $[4\ 5\ 4\ 3\ 4\ 2] \rightarrow R_2 = [2\ 3\ 4\ 4\ 4\ 5]$
- 3 $[1\ 5\ 2\ 1\ 3] \rightarrow R_3 = [1\ 1\ 2\ 3\ 5]$

阶段 2: 多路归并 (Multi-Way Merge)

将 $\lceil B(R)/M \rceil$ 个归并段中的元组进行归并

过程: 多路归并

- 1: 将每个归并段文件的第一页读入缓冲池的一页
- 2: **repeat**
- 3: 找出所有输入缓冲页中最小的排序键值 v
- 4: 将所有排序键值等于 v 的元组写入输出缓冲区
- 5: 当一个输入缓冲页中的元组用尽时, 读入其归并段的下一页
- 6: **until** 所有归并段全都归并完毕



阶段 2: 多路归并

归并段	内存页	归并段文件剩余页
R_1	1 2	2 2 2 5
R_2	2 3	4 4 4 5
R_3	1 1	2 3 5

输出: 1 1 1 (红色的数代表刚被输出的元组)

归并段	内存页	归并段文件剩余页
R_1	2	2 2 2 5
R_2	2 3	4 4 4 5
R_3	2 3	5

输出: 1 1 1 2 2

阶段 2：多路归并

	归并段	内存页	归并段文件剩余页
最小排序键 = 2	R_1	2 2	2 5
	R_2	3	4 4 4 5
	R_3	3	5

输出: 1 1 1 2 2 2 2 2

	归并段	内存页	归并段文件剩余页
最小排序键 = 2	R_1	2 5	
	R_2	3	4 4 4 5
	R_3	3	5

输出: 1 1 1 2 2 2 2 2 2

阶段 2：多路归并

	归并段	内存页	归并段文件剩余页
最小排序键 = 3	R_1	5	
	R_2	3	4 4 4 5
	R_3	3	5

输出: 1 1 1 2 2 2 2 2 2 3 3

	归并段	内存页	归并段文件剩余页
最小排序键 = 4	R_1	5	
	R_2	4 4	4 5
	R_3	5	

输出: 1 1 1 2 2 2 2 2 2 3 3 4 4

阶段 2：多路归并

	归并段	内存页	归并段文件剩余页
最小排序键 = 4	R_1	5	
	R_2	4 5	
	R_3	5	

输出: 1 1 1 2 2 2 2 2 2 3 3 4 4 4

	归并段	内存页	归并段文件剩余页
最小排序键 = 5	R_1	5	
	R_2	5	
	R_3	5	

输出: 1 1 1 2 2 2 2 2 2 3 3 4 4 5 5 5 算法结束

算法分析的指标

- I/O 次数
- 可用内存页数 M 的最小值

算法分析的注意事项

分析算法时不考虑结果输出操作

输出结果产生的 I/O 不计入算法的 I/O

- 输出结果可能直接作为后续操作的输入，无需写入文件

输出缓冲区不计入可用内存

- 如果输出结果直接作为后续操作的输入，那么输出缓冲区将计入后续操作的可用内存页数

算法分析

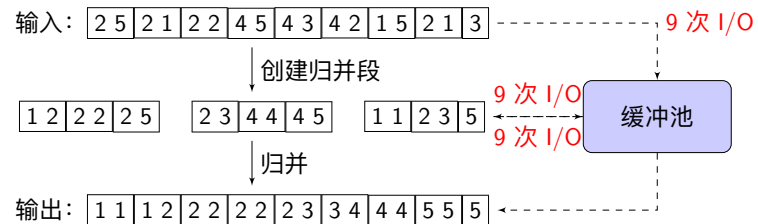
I/O 次数 = $3B(R)$

- 创建归并段时，读 R 的每个页仅 1 次，合计 $B(R)$ 次 I/O
- 将每个归并段写入文件，合计 $B(R)$ 次 I/O
- 归并阶段，扫描每个归并段仅 1 次，合计 $B(R)$ 次 I/O

可用内存页数 $M \geq \sqrt{B(R)}$

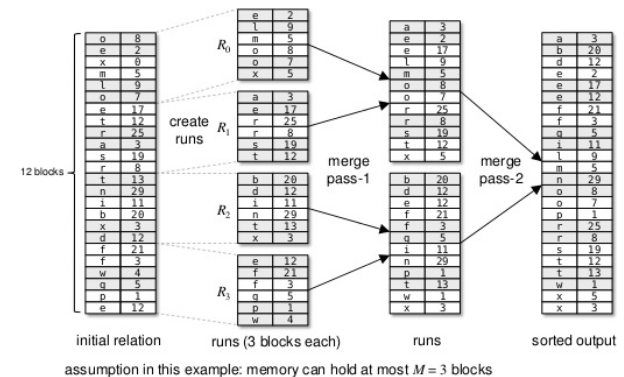
- 每个归并段不超过 M 页
- 最多 M 个归并段

算法分析



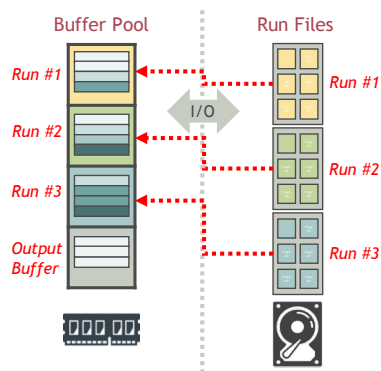
多趟多路外存归并排序 (Multi-Pass Multi-Way External Merge Sort)

- 若 $B(R) > M^2$ ，则需要执行多趟多路外存归并排序
- I/O 次数 = $(2m - 1)B(R)$ ，其中 m 算法执行趟数



多路归并排序的优化

- 当某缓冲页中所有元组都被归并完毕时，DBMS 需要读入其归并段的下一页
- 此时，归并过程被迫等待，直至 I/O 完成



双缓冲 (Double Buffering)

为每个归并段分配多个内存页作为输入缓冲区，并组成环形 (circular)

- 在当前缓冲页中所有元组都已归并完毕时，DBMS 直接开始归并下一缓冲页中的元组
- 与此同时，DBMS 将归并段文件的下一块读入空闲的缓冲页

归并段	内存页	归并段文件剩余页
R_1	1 2 2 2	2 5
R_2	2 3 4 4	4 5
R_3	2 3 1 1	5

☒ 正在被归并的页 ☐ 缓冲的页 (Double Page)

6.2.2 选择算子

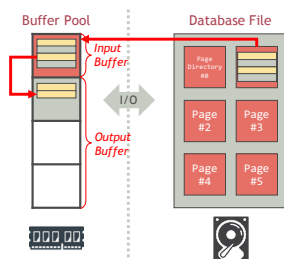
选择算子的执行算法

- 基于扫描的选择算法 (Scanning-based Selection)
- 基于哈希的选择算法 (Hash-based Selection)
- 基于索引的选择算法 (Index-based Selection)

基于扫描的选择算法

算法

- 1: for R 文件的每个页 P do
- 2: 将 P 读入缓冲区 (通过缓冲池管理器)
- 3: for P 中每条元组 t do
- 4: if t 满足选择条件 then
- 5: 输出 t (先写入输出缓冲区, 待输出缓冲区写满后, 刷写到输出结果文件)



算法分析

I/O 次数 = $B(R)$

- R 文件的每个页仅读 1 次

可用内存页数 $M \geq 1$

- 至少 1 个页作为 R 的输入缓冲区

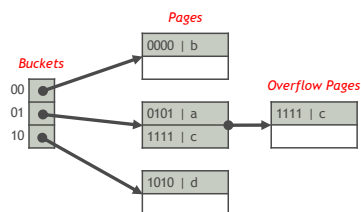
基于哈希的选择算法

使用该算法的前提条件

- 关系 R 采用哈希文件组织
- K 是 R 的哈希键 (Hash Key)
- 选择条件形如 $K = v$

算法

- 1: 根据 $hash(v)$ 确定结果元组所在的桶
- 2: 在桶的页链表中搜索 $K = v$ 的元组, 将其输出



算法分析

I/O 次数 $\approx \lceil B(R)/V(R, K) \rceil$

- 属性 K 有 $V(R, K)$ 个不同的值
- 每个桶平均有 $\lceil B(R)/V(R, K) \rceil$ 个页 (不准确的估计)

可用内存页数 $M \geq 1$

- 至少 1 个页作为输入缓冲区, 用于扫描桶的页链表

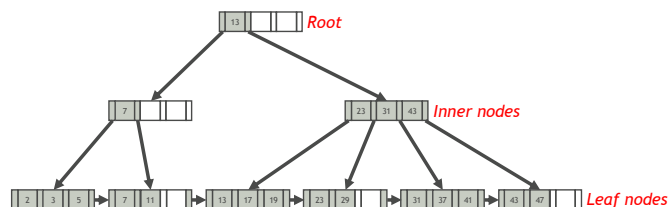
基于索引的选择算法

使用该算法的前提条件

- 选择条件形如 $K = v$ 或 $l \leq K \leq u$
- 关系 R 上建有属性 K 上的索引

算法

- 1: 在索引上查找满足选择条件的元组的 RID
- 2: 根据元组 RID, 通过缓冲池管理器读这些元组, 并输出



算法分析

I/O 代价 $\approx [B(R)/V(R, K)]$ (索引是聚簇索引)

- 结果元组连续存储于文件中
- 属性 K 有 $V(R, K)$ 个不同的值
- 结果元组约占 $[B(R)/V(R, K)]$ 个页 (不准确的估计)

I/O 代价 $\approx [T(R)/V(R, K)]$ (索引是非聚簇索引)

- 约有 $[T(R)/V(R, K)]$ 个结果元组 (不准确的估计)
- 结果元组不一定连续存储于文件中
- 最坏情况下, 所有结果元组均在不同的页上

可用内存页数 $M \geq 1$

- 至少 1 页作为输入缓冲区, 用于读 B+ 树节点

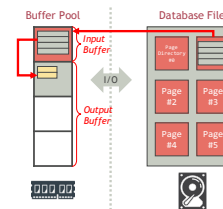
6.2.3 投影算子

不含去重的投影算法

算法

- 1: **for** R 文件的每个页 P **do**
- 2: 将 P 读入缓冲区
- 3: **for** P 中每条元组 t **do**
- 4: 输出 t 的投影元组

该算法在数据访问模式 (Access Pattern) 上与基于扫描的选择算法相同



算法分析

I/O 次数 = $B(R)$

- R 文件的每个页仅读 1 次

可用内存页数 $M \geq 1$

- 至少 1 个页作为 R 的输入缓冲区

6.2.4 去重算子

去重算子 (Duplicate Elimination)

关系 R 上的去重算子 $\delta(R)$ 返回 R 中互不相同的元组

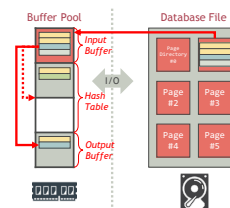
- ① 一趟去重算法 (One-Pass Duplicate Elimination)
- ② 基于排序的去重算法 (Sort-based Duplicate Elimination)
- ③ 基于哈希的去重算法 (Hash-based Duplicate Elimination)

一趟去重算法

算法

- 1: for R 文件的每个页 P do
- 2: 将 P 读入缓冲区
- 3: for P 中每条元组 t do
- 4: if 未见过 t then
- 5: 输出 t

除 R 的输入缓冲区外，在其余可用缓冲区中建立内存哈希表，记录见过的元组（哈希键为元组本身）



一趟去重算法

例 (一趟去重算法)

- $R = [2\ 5\ 2\ 1\ 2\ 2\ 4\ 5\ 4\ 3\ 4\ 2\ 1\ 5\ 2\ 1\ 3]$
- $M = 4$

输入缓冲区 (1 页)	内存哈希表 (< M 页)	输出
2 5	2 5	2 5
2 1	2 5 1	2 5 1
2 2	2 5 1	2 5 1
4 5	2 5 1 4	2 5 1 4
4 3	2 5 1 4 3	2 5 1 4 3
4 2	2 5 1 4 3	2 5 1 4 3
1 5	2 5 1 4 3	2 5 1 4 3
2 1	2 5 1 4 3	2 5 1 4 3
3	2 5 1 4 3	2 5 1 4 3

算法分析

该算法在数据访问模式上与基于扫描的选择算法相同

I/O 代价 = $B(R)$

- R 的每个页仅读 1 次

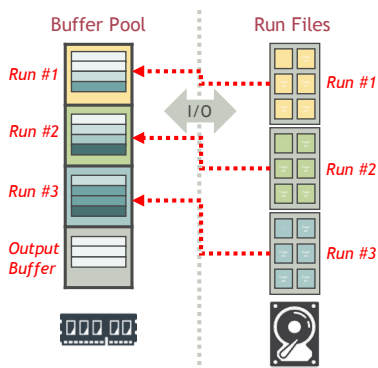
可用内存页数 $M \geq B(\delta(R)) + 1$

- 必要条件: R 中互不相同的元组 $\delta(R)$ 能在 $M - 1$ 页缓冲区中存得下

基于排序的去重算法

与多路外归并排序算法基本相同, 两点区别如下:

- 创建归并段时, 以整个元组为排序键, 进行排序
- 在归并阶段, 相同的元组只输出 1 个, 其余全部丢弃



基于排序的去重算法

例 (基于排序的去重算法)

- $R = [2\ 5\ 2\ 1\ 2\ 2\ 4\ 5\ 4\ 3\ 4\ 2\ 1\ 5\ 2\ 1\ 3]$
- $M = 3$

创建归并段

- $R_1 = [1\ 2\ 2\ 2\ 2\ 5]$
- $R_2 = [2\ 3\ 4\ 4\ 4\ 5]$
- $R_3 = [1\ 1\ 2\ 3\ 5]$

基于排序的去重算法

	归并段	内存页	归并段文件剩余页
最小元组 = 1	R ₁	1 2	2 2 2 5
	R ₂	2 3	4 4 4 5
	R ₃	1 1	2 3 5

输出: 1 (红色的数代表刚输出的去重组)

	归并段	内存页	归并段文件剩余页
最小元组 = 2	R ₁	2	2 2 2 5
	R ₂	2 3	4 4 4 5
	R ₃	2 3	5

输出: 1 2

基于排序的去重算法

	归并段	内存页	归并段文件剩余页
最小元组 = 2	R ₁	2 2	2 5
	R ₂	3	4 4 4 5
	R ₃	3	5

输出: 1 2

	归并段	内存页	归并段文件剩余页
最小元组 = 2	R ₁	2 5	
	R ₂	3	4 4 4 5
	R ₃	3	5

输出: 1 2

基于排序的去重算法

	归并段	内存页	归并段文件剩余页
最小元组 = 3	R ₁	5	
	R ₂	3	4 4 4 5
	R ₃	3	5

输出: 1 2 3

	归并段	内存页	归并段文件剩余页
最小元组 = 4	R ₁	5	
	R ₂	4 4	4 5
	R ₃	5	

输出: 1 2 3 4

基于排序的去重算法

	归并段	内存页	归并段文件剩余页
最小元组 = 4	R ₁	5	
	R ₂	4 5	
	R ₃	5	

输出: 1 2 3 4

	归并段	内存页	归并段文件剩余页
最小元组 = 5	R ₁	5	
	R ₂	5	
	R ₃	5	

输出: 1 2 3 4 5 算法结束

算法分析

I/O 次数 = $3B(R)$

- 创建归并段时，读 R 的每个页仅 1 次，合计 $B(R)$ 次 I/O
- 将每个归并段写入文件，合计 $B(R)$ 次 I/O
- 归并阶段，扫描每个归并段仅 1 次，合计 $B(R)$ 次 I/O

可用内存页数 $M \geq \sqrt{B(R)}$

- 每个归并段不超过 M 页
- 最多 M 个归并段

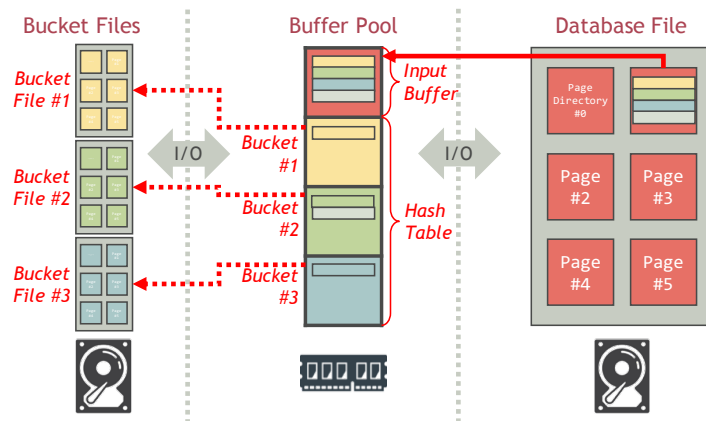
基于哈希的去重算法

算法

- 1: // 阶段 1: 哈希分桶
- 2: **for** R 文件的每个页 P **do**
- 3: 将 P 读入缓冲区
- 4: 将其余 $M-1$ 个缓冲区页分别作为 $M-1$ 个桶 $R_1, R_2, \dots, R_{M-1}$ 的输出缓冲区
- 5: 将 P 中元组分配到 $R_1, R_2, \dots, R_{M-1}$ 中 (哈希键为整个元组)
- 6: // 阶段 2: 逐桶去重
- 7: **for** $i = 1, 2, \dots, M-1$ **do**
- 8: 在 R_i 上执行一趟去重算法，并输出结果

阶段 1: 哈希分桶

重复的元组必落入相同的桶中



阶段 2: 逐桶去重

每个桶 R_i 的去重结果合并在一起就得到了 R 的去重结果

- $\delta(R) = \bigcup_{i=1}^{M-1} \delta(R_i)$
- 对任意 $i \neq j$, $\delta(R_i) \cap \delta(R_j) = \emptyset$

基于哈希的去重算法

例 (基于哈希的去重算法)

- $R = [2\ 5\ 2\ 1\ 2\ 2\ 4\ 5\ 4\ 3\ 4\ 2\ 1\ 5\ 2\ 1\ 3]$
- $M = 4$
- $h(K) = K \bmod 3$

哈希分桶

- $R_0 = [3\ 3]$
- $R_1 = [1\ 4\ 4\ 4\ 1\ 1]$
- $R_2 = [2\ 5\ 2\ 2\ 2\ 5\ 2\ 5\ 2]$

逐桶去重

- $\delta(R_0) = [3]$
- $\delta(R_1) = [1\ 4]$
- $\delta(R_2) = [2\ 5]$

算法分析

I/O 次数 = $3B(R)$

- 哈希分桶时, R 文件的每个页仅读 1 次, 合计 $B(R)$ 次 I/O
- 将每个桶写入各自的文件, 合计 $\sum_{i=1}^{M-1} B(R_i) \approx B(R)$ 次 I/O
- 在每个桶 R_i 上执行一趟去重算法的 I/O 代价是 $B(R_i)$

可用内存页数 $M \geq \sqrt{B(R)} + 1$

- 共 $M - 1$ 个桶
- 必要条件: 如果每个桶 R_i 不超过 $M - 1$ 页, 则在 R_i 上执行一趟去重算法时, 满足可用内存页数要求

6.2.5 聚集算子

聚集算子 (Aggregation Operations)

聚集算子和去重算子的实现算法非常相似, 自己独立思考并设计算法

- ① 一趟聚集算法 (One-Pass Aggregation)
- ② 基于排序的聚集算法 (Sort-based Aggregation)
- ③ 基于哈希的聚集算法 (Hash-based Aggregation)

6.2.6 集合算子

集合差算子

- ① 一趟集合差算法 (One-Pass Set Difference)
- ② 基于排序的集合差算法 (Sort-based Set Difference)
- ③ 基于哈希的集合差算法 (Hash-based Set Difference)

一趟集合差算法

一趟集合差算法

算法

1: // 构建 (Build) 阶段

2: 在 $M - 1$ 个缓冲区页中建立内存查找结构 (哈希表或平衡二叉树), 查找键是整个元组

3: for S 文件的每个页 P do

4: 将 P 读入缓冲区

5: 将 P 中元组插入内存查找结构

6: // 探测 (Probe) 阶段

7: for R 文件的每个页 P do

8: 将 P 读入缓冲区

9: for P 中每条元组 t do

10: if t 不在于内存查找结构中 then

11: 输出 t

例 (一趟集合差算法)

- $R =$

1	5	8	2	3	10	4	7	6	9
---	---	---	---	---	----	---	---	---	---
- $S =$

4	11	9	5	7	3	6	12	8	10
---	----	---	---	---	---	---	----	---	----
- $M = 6$

缓冲区前 $M - 1$ 页中为 S 元组建立的内存哈希表:

4	11	9	5	7	3	6	12	8	10
---	----	---	---	---	---	---	----	---	----

序号	缓冲区第 M 页	输出
1	1 5	1
2	8 2	1 2
3	3 10	1 2
4	4 7	1 2
5	6 9	1 2

算法分析

I/O 次数 = $B(R) + B(S)$

- 构建 (Build) 阶段, S 文件的每个页仅读 1 次, 合计 $B(S)$ 次 I/O
- 探测 (Probe) 阶段, R 文件的每个页仅读 1 次, 合计 $B(R)$ 次 I/O

可用内存页数 $M \geq B(S) + 1$

- 内存查找结构约占 $B(S)$ 页

基于哈希的集合差算法

算法

- // 哈希分桶 (与基于哈希的去重算法的哈希分桶方法相同)
- 将 R 的元组分配到 $M - 1$ 个桶 $R_1, R_2, \dots, R_{M-1}$ 中 (哈希键为整个元组)
- 将 S 的元组分配到 $M - 1$ 个桶 $S_1, S_2, \dots, S_{M-1}$ 中 (哈希键为整个元组)
- // 逐桶计算集合差
- for $i = 1, 2, \dots, M - 1$ do
- 使用一趟集合差算法计算 $R_i - S_i$, 并输出结果

- R 和 S 中相同的元组一定分别落入相同编号的桶 R_i 和 S_i 中
- $R - S = \bigcup_{i=1}^{M-1} (R_i - S_i)$
- 对任意 $i \neq j$, $(R_i - S_i) \cap (R_j - S_j) = \emptyset$

基于哈希的集合差算法

例 (基于哈希的集合差算法)

- $R = \begin{bmatrix} 1 & 5 \\ 8 & 2 \\ 3 & 10 \\ 4 & 7 \\ 6 & 9 \end{bmatrix}$
- $S = \begin{bmatrix} 4 & 11 \\ 9 & 5 \\ 7 & 3 \\ 6 & 12 \\ 8 & 10 \end{bmatrix}$
- $M = 4$
- $h(K) = K \bmod 3$

R 的桶

- $R_0 = \begin{bmatrix} 3 & 6 \\ 9 \end{bmatrix}$
- $R_1 = \begin{bmatrix} 1 & 10 \\ 4 & 7 \end{bmatrix}$
- $R_2 = \begin{bmatrix} 5 & 2 \\ 8 \end{bmatrix}$

S 的桶

- $S_0 = \begin{bmatrix} 9 & 3 \\ 6 & 12 \end{bmatrix}$
- $S_1 = \begin{bmatrix} 4 & 7 \\ 10 \end{bmatrix}$
- $S_2 = \begin{bmatrix} 11 & 5 \\ 8 \end{bmatrix}$

集合差

- $R_0 - S_0 = \emptyset$
- $R_1 - S_1 = \begin{bmatrix} 1 \\ 10 \end{bmatrix}$
- $R_2 - S_2 = \begin{bmatrix} 2 \end{bmatrix}$

算法分析

I/O 代价 = $3B(R) + 3B(S)$

- 对 R 进行哈希分桶时, R 文件的每个页仅读 1 次, 合计 $B(R)$ 次 I/O
- 将 R 的每个桶写入各自的文件, 共需 $\sum_{i=1}^{M-1} B(R_i) \approx B(R)$ 次 I/O
- 对 S 进行哈希分桶时, S 文件的每个页仅读 1 次, 合计 $B(S)$ 次 I/O
- 将 S 的每个桶写入各自的文件, 共需 $\sum_{i=1}^{M-1} B(S_i) \approx B(S)$ 次 I/O
- 使用一趟集合差算法计算 $R_i - S_i$ 的 I/O 代价是 $B(R_i) + B(S_i)$

可用内存页数 $M \geq \sqrt{B(S)} + 1$

- S 共有 $M - 1$ 个桶
- S 的每个桶不超过 $M - 1$ 页

基于排序的集合差算法

算法

```
1: // 创建归并段
2: 将  $R$  划分为  $\lceil B(R)/M \rceil$  个归并段 (每个归并段按整个元组进行排序)
3: 将  $S$  划分为  $\lceil B(S)/M \rceil$  个归并段 (每个归并段按整个元组进行排序)
4: // 归并
5: 读入  $R$  和  $S$  的每个归并段的第 1 页
6: repeat
7:   找出输入缓冲区中最小的元组  $t$ 
8:   if  $t \in R$  且  $t \notin S$  then
9:     将  $t$  写入输出缓冲区
10:  从输入缓冲区中删除  $t$  的所有副本
11:  任意输入缓冲页中的元组若归并完毕, 则读入其归并段的下一页
12: until  $R$  的所有归并段都已归并完毕
```

基于排序的集合差算法

例 (基于排序的集合差算法)

- $R =$

1	5	8	2	3	10	4	7	6	9
---	---	---	---	---	----	---	---	---	---
- $S =$

4	11	9	5	7	3	6	12	8	10
---	----	---	---	---	---	---	----	---	----
- $M = 4$

R 的归并段

- $R_0 =$

1	2	3	4	5	7	8	10
---	---	---	---	---	---	---	----
- $R_1 =$

6	9
---	---

S 的归并段

- $S_0 =$

3	4	5	6	7	8	9	11
---	---	---	---	---	---	---	----
- $S_1 =$

8	10
---	----

算法分析

I/O 次数 = $3B(R) + 3B(S)$

- 对 R 创建归并段时, R 文件的每个页仅读 1 次, 合计 $B(R)$ 次 I/O
- 将 R 的每个归并段写入各自的文件, 共需 $B(R)$ 次 I/O
- 对 S 创建归并段时, S 文件的每个页仅读 1 次, 合计 $B(S)$ 次 I/O
- 将 S 的每个归并段写入各自的文件, 共需 $B(S)$ 次 I/O
- 归并阶段, 对 R 和 S 的每个归并段仅扫描 1 次, 合计 $B(R) + B(S)$ 次 I/O

可用内存页数 $M \geq \sqrt{B(R) + B(S)}$

- R 和 S 的每个归并段均不超过 M 页
- R 和 S 共有不超过 M 个归并段

自主学习: 集合并算子

集合并算子和集合差算子的实现算法基本相同, 仅在部分细节上有差异

- ① 一趟集合并算法
- ② 基于排序的集合并算法
- ③ 基于哈希的集合并算法

自主学习：集合交算子

集合交算子和集合差算子的实现算法基本相同，仅在部分细节上有差异

- ① 一趟集合交算法
- ② 基于排序的集合交算法
- ③ 基于哈希的集合交算法

6.2.7 连接算子

连接算子 (Join)

下面以 $R(X, Y) \bowtie S(Y, Z)$ 为例，介绍连接算子的实现算法，并假设 $B(S) \leq B(R)$

- ① 一趟连接算法 (One-Pass Join)
- ② 嵌套循环连接算法 (Nested-Loop Join)
- ③ 排序归并连接算法 (Sort-Merge Join)
- ④ 哈希连接算法 (Hash Join)
- ⑤ 索引连接算法 (Index Join)

一趟连接算法

算法

1: // 构建 (Build) 阶段

2: 在 $M - 1$ 个缓冲区页中建立内存查找结构 (哈希表或平衡二叉树)，查找键是 $S.Y$

3: **for** S 文件的每个页 P **do**

4: 将 P 读入缓冲区

5: 将 P 中元组插入内存查找结构

6: // 探测 (Probe) 阶段

7: **for** R 文件的每个页 P **do**

8: 将 P 读入缓冲区

9: **for** P 中每条元组 r **do**

10: **for** 内存查找结构中每条键值等于 $r.Y$ 的元组 s **do**

11: 输出 r 和 s 的连接元组

一趟连接算法

例 (一趟连接算法)

- $R(X, Y) = \begin{array}{|c|c|c|} \hline (1, 1), (5, 5) & (3, 2), (3, 1) & (2, 1), (4, 2) \\ \hline \end{array}$
- $S(Y, Z) = \begin{array}{|c|c|c|} \hline (2, 6), (1, 7) & (1, 8), (2, 5) & (2, 7) \\ \hline \end{array}$
- $M = 4$

缓冲区前 $M - 1$ 页中为 S 元组建立的内存哈希表: $\begin{array}{|c|c|c|} \hline (2, 6), (1, 7) & (1, 8), (2, 5) & (2, 7) \\ \hline \end{array}$

序号	缓冲区第 M 页	输出
1	$(1, 1), (5, 5)$	$(1, 1, 7), (1, 1, 8)$
2	$(3, 2), (3, 1)$	$\dots, (3, 2, 6), (3, 2, 5), (3, 2, 7), (3, 1, 7), (3, 1, 8)$
3	$(2, 1), (4, 2)$	$\dots, (2, 1, 7), (2, 1, 8), (4, 2, 6), (4, 2, 5), (4, 2, 7)$

算法分析

I/O 代价 = $B(R) + B(S)$

- 构建 (Build) 阶段, S 文件的每个页仅读 1 次, 合计 $B(S)$ 次 I/O
- 探测 (Probe) 阶段, R 文件的每个页仅读 1 次, 合计 $B(R)$ 次 I/O

可用内存页数 $M \geq B(S) + 1$

- 内存查找结构约占 $B(S)$ 页

基于块的嵌套循环连接算法 (Block-based Nested-Loop Join)

- 小关系 S 在外层循环, 叫作外关系 (Outer Relation)
- 大关系 S 在内层循环, 叫作内关系 (Inner Relation)

算法

```
1: for  $S$  文件的每  $M - 1$  个页 do
2:   将这  $M - 1$  个页读入缓冲区
3:   建立一个内存查找结构, 组织这  $M - 1$  个页中的元组
4:   for  $R$  文件的每个页  $P$  do
5:     将  $P$  读入缓冲区
6:     for  $P$  中每条元组  $r$  do
7:       for 内存查找结构中能与  $r$  进行连接的元组  $s$  do
8:         输出  $r$  和  $s$  的连接元组
```

基于块的嵌套循环连接算法

例 (基于块的嵌套循环连接算法)

- $R(X, Y) = \begin{array}{|c|c|c|} \hline (1, 1), (5, 5) & (3, 2), (3, 1) & (2, 1), (4, 2) \\ \hline \end{array}$
- $S(Y, Z) = \begin{array}{|c|c|c|} \hline (2, 6), (1, 7) & (1, 8), (2, 5) & (2, 7) \\ \hline \end{array}$
- $M = 3$

缓冲区前 $M - 1$ 页	缓冲区第 M 页	输出
$(2, 6), (1, 7)$	$(1, 8), (2, 5)$	$(1, 1, 7), (1, 1, 8)$
$(2, 6), (1, 7)$	$(1, 8), (2, 5)$	$(3, 2, 6), (3, 2, 5), (3, 1, 7), (3, 1, 8)$
$(2, 6), (1, 7)$	$(1, 8), (2, 5)$	$(2, 1, 7), (2, 1, 8), (4, 2, 6), (4, 2, 5)$
$(2, 7)$	$(1, 1), (5, 5)$	
$(2, 7)$	$(3, 2), (3, 1)$	$(3, 2, 7)$
$(2, 7)$	$(2, 1), (4, 2)$	$(4, 2, 7)$

算法分析

I/O 次数 = $B(S) + \frac{B(R)B(S)}{M-1}$

- 外关系 S 文件的每个页仅读 1 次，合计 B(S) 次 I/O
- 内关系 R 文件被扫描 B(S)/(M-1) 次，合计 $\frac{B(R)B(S)}{M-1}$ 次 I/O

可用内存页数 $M \geq 2$

- 至少 1 页作为 S 的输入缓冲区
- 1 页作为 R 的输入缓冲区

排序归并连接 (Sort-Merge Join)

算法

- // 创建归并段
- 将 R 划分为 $\lceil B(R)/M \rceil$ 个归并段 (每个归并段的元组按 R.Y 排序)
- 将 S 划分为 $\lceil B(S)/M \rceil$ 个归并段 (每个归并段的元组按 S.Y 排序)
- // 归并
- 读入 R 和 S 的每个归并段的第 1 页
- repeat
- 找出输入缓冲区中元组 Y 属性的最小值 y
- for R 中满足 R.Y = y 的元组 r do
- for S 中满足 S.Y = y 的元组 s do
- 连接 r 和 s，并将结果写入输出缓冲区
- 任意输入缓冲页中的元组若归并完毕，则读入其归并段的下一页
- until R 或 S 的所有归并段都已归并完毕

排序归并连接算法

例 (排序归并连接算法)

- $R(X, Y) =$

(1, 1)	(5, 4)	(3, 2)	(3, 1)	(6, 3)	(2, 1)	(4, 2)	(8, 5)	(4, 1)	(3, 4)
--------	--------	--------	--------	--------	--------	--------	--------	--------	--------
- $S(Y, Z) =$

(2, 6)	(1, 7)	(1, 8)	(5, 9)	(5, 3)	(2, 5)	(3, 1)	(2, 7)	(3, 7)	(4, 9)
--------	--------	--------	--------	--------	--------	--------	--------	--------	--------
- M = 4

创建归并段

- $R_1 =$

(1, 1)	(2, 1)	(3, 1)	(3, 2)	(6, 3)	(5, 4)
--------	--------	--------	--------	--------	--------
- $R_2 =$

(4, 1)	(4, 2)	(3, 4)	(8, 5)
--------	--------	--------	--------
- $S_1 =$

(1, 7)	(1, 8)	(2, 5)	(2, 6)	(5, 3)	(5, 9)
--------	--------	--------	--------	--------	--------
- $S_2 =$

(2, 7)	(3, 1)	(3, 7)	(4, 9)
--------	--------	--------	--------

排序归并连接算法

归并段	内存页	归并段文件剩余页	
R_1	(1, 1), (2, 1)	(3, 1), (3, 2)	(6, 3), (5, 4)
R_2	(4, 1), (4, 2)	(3, 4), (8, 5)	
S_1	(1, 7), (1, 8)	(2, 5), (2, 6)	(5, 3), (5, 9)
S_2	(2, 7), (3, 1)	(3, 7), (4, 9)	

归并段	内存页	归并段文件剩余页	
R_1	(1, 1), (2, 1)	(3, 1), (3, 2)	(6, 3), (5, 4)
R_2	(4, 1), (4, 2)	(3, 4), (8, 5)	
S_1	(1, 7), (1, 8)	(2, 5), (2, 6)	(5, 3), (5, 9)
S_2	(2, 7), (3, 1)	(3, 7), (4, 9)	

输出: (1, 1, 7), (1, 1, 8), (2, 1, 7), (2, 1, 8), (3, 1, 7), (3, 1, 8), (4, 1, 7), (4, 1, 8)

排序归并连接算法

	归并段	内存页	归并段文件剩余页
最小连接键 = 2	R_1	(3, 2)	(6, 3), (5, 4)
	R_2	(4, 2)	(3, 4), (8, 5)
	S_1	(2, 5), (2, 6)	(5, 3), (5, 9)
	S_2	(2, 7), (3, 1)	(3, 7), (4, 9)

	归并段	内存页	归并段文件剩余页
最小连接键 = 2	R_1	(3, 2) (6, 3), (5, 4)	
	R_2	(4, 2) (3, 4), (8, 5)	
	S_1	(2, 5), (2, 6) (5, 3), (5, 9)	
	S_2	(2, 7), (3, 1) (3, 7), (4, 9)	

输出: ..., (3, 2, 5), (3, 2, 6), (3, 2, 7), (4, 2, 5), (4, 2, 6), (4, 2, 7)

排序归并连接算法

	归并段	内存页	归并段文件剩余页
最小连接键 = 3	R_1	(6, 3), (5, 4)	
	R_2	(3, 4), (8, 5)	
	S_1	(5, 3), (5, 9)	
	S_2	(3, 1) (3, 7), (4, 9)	

	归并段	内存页	归并段文件剩余页
最小连接键 = 3	R_1	(6, 3), (5, 4)	
	R_2	(3, 4), (8, 5)	
	S_1	(5, 3), (5, 9)	
	S_2	(3, 1) (3, 7), (4, 9)	

输出: ..., (6, 3, 1), (6, 3, 7)

排序归并连接算法

	归并段	内存页	归并段文件剩余页
最小连接键 = 4	R_1	(5, 4)	
	R_2	(3, 4), (8, 5)	
	S_1	(5, 3), (5, 9)	
	S_2	(4, 9)	

输出: ..., (5, 4, 9), (3, 4, 9)

	归并段	内存页	归并段文件剩余页
最小连接键 = 5	R_1		
	R_2	(8, 5)	
	S_1	(5, 3), (5, 9)	
	S_2		

输出: ..., (8, 5, 3), (8, 5, 9) 算法结束

算法分析

I/O 次数 = $3B(R) + 3B(S)$

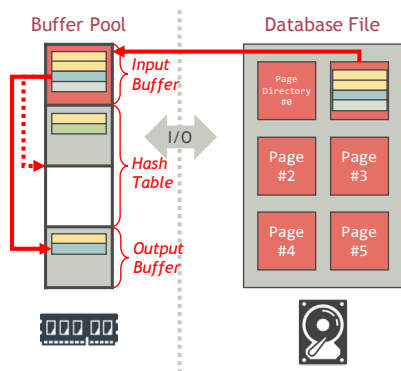
- 对 R 创建归并段时, R 文件的每个页仅读 1 次, 合计 $B(R)$ 次 I/O
- 将 R 的每个归并段写入各自的文件, 共需 $B(R)$ 次 I/O
- 对 S 创建归并段时, S 文件的每个页仅读 1 次, 合计 $B(S)$ 次 I/O
- 将 S 的每个归并段写入各自的文件, 共需 $B(S)$ 次 I/O
- 归并阶段, 对 R 和 S 的每个归并段仅扫描 1 次, 合计 $B(R) + B(S)$ 次 I/O

可用内存页数 $M \geq \sqrt{B(R) + B(S)}$

- R 和 S 的每个归并段均不超过 M 页
- R 和 S 共有不超过 M 个归并段

经典哈希连接 (Classic Hash Join)

如果一趟连接算法使用的内存查找结构是哈希表, 则该算法称作经典哈希连接算法



Grace 哈希连接 (Grace Hash Join)

算法

- 1: // 哈希分桶
- 2: 将 R 的元组分配到 $M-1$ 个桶 $R_1, R_2, \dots, R_{M-1}$ 中 (哈希键为 $R.Y$)
- 3: 将 S 的元组分配到 $M-1$ 个桶 $S_1, S_2, \dots, S_{M-1}$ 中 (哈希键为 $S.Y$)
- 4: // 逐桶连接
- 5: **for** $i = 1, 2, \dots, M-1$ **do**
- 6: 使用一趟连接算法计算 $R_i \bowtie S_i$, 并输出结果

- R 和 S 中相同的元组一定分别落入同号桶 R_i 和 S_i 中
- $R \bowtie S = \bigcup_{i=1}^{M-1} (R_i \bowtie S_i)$
- 对任意 $i \neq j$, $(R_i \bowtie S_i) \cap (R_j \bowtie S_j) = \emptyset$

算法分析

I/O 次数 = $3B(R) + 3B(S)$

- 对 R 进行哈希分桶时, R 文件的每个页仅读 1 次, 合计 $B(R)$ 次 I/O
- 将 R 的桶全部写入文件, 共需 $\sum_{i=1}^{M-1} B(R_i) \approx B(R)$ 次 I/O
- 对 S 进行哈希分桶时, S 文件的每个页仅读 1 次, 合计 $B(S)$ 次 I/O
- 将 S 的桶全部写入文件, 共需 $\sum_{i=1}^{M-1} B(S_i) \approx B(S)$ 次 I/O
- 使用一趟连接算法计算 $R_i \bowtie S_i$ 的 I/O 代价是 $B(R_i) + B(S_i)$

可用内存页数 $M \geq \sqrt{B(S)} + 1$

- S 共有 $M-1$ 个桶
- S 的每个桶不超过 $M-1$ 页

索引连接 (Index Join)

设关系 S 上建有属性 $S.Y$ 上的索引

算法

- 1: **for** R 文件的每个页 P **do**
- 2: 将 P 读入缓冲区
- 3: **for** P 中每条元组 r **do**
- 4: 在索引上查找键值等于 $r.Y$ 的 S 的元组集合 T
- 5: **for** $s \in T$ **do**
- 6: 输出 r 和 s 的连接元组

索引连接算法

例 (索引连接)

- $R(X, Y) = \begin{bmatrix} (1, 1), (5, 5) \\ (3, 2), (3, 1) \\ (2, 1), (4, 2) \end{bmatrix}$
- $S(Y, Z) = \begin{bmatrix} (2, 6), (1, 7) \\ (1, 8), (2, 5) \\ (2, 7) \end{bmatrix}$ ($S.Y$ 上有索引)
- $M = 2$

序号	R 的输入缓冲区	输出
1	$(1, \textcolor{red}{1}), (5, 5)$	$(1, 1, 7), (1, 1, 8)$
2	$(3, \textcolor{red}{2}), (3, \textcolor{red}{1})$	$\dots, (3, 2, 6), (3, 2, 5), (3, 2, 7), (3, 1, 7), (3, 1, 8)$
3	$(2, \textcolor{red}{1}), (4, \textcolor{red}{2})$	$\dots, (2, 1, 7), (2, 1, 8), (4, 2, 6), (4, 2, 5), (4, 2, 7)$

算法分析

I/O 次数 = $B(R) + \frac{T(R)T(S)}{V(S,Y)}$ (索引是非聚簇索引)

- R 文件的每个页仅读 1 次, 合计 $B(R)$ 次 I/O
- 对于 R 的每个元组 r , S 中平均约有 $T(S)/V(S, Y)$ 个元组能与 r 连接
- 因为是非聚簇索引, 所以这些元组在 S 文件中不一定连续存储。最坏情况下, 读每个元组会产生 1 次 I/O, 合计 $\frac{T(R)T(S)}{V(S,Y)}$ 次 I/O

I/O 次数 = $B(R) + T(R) \lceil \frac{B(S)}{V(S,Y)} \rceil$ (索引是聚簇索引)

- 因为是聚簇索引, 所以对于 R 的每个元组 r , S 中能与 r 连接的元组一定连续存储在 S 文件中, 约占 $\lceil \frac{B(S)}{V(S,Y)} \rceil$ 个页

可用内存页数 $M \geq 2$

- 1 页作为 R 的输入缓冲区
- 1 页作为索引的输入缓冲区

6.3 查询执行模型

查询计划的执行方法

如何执行由多个算子构成的查询计划?

- ① 物化执行 (Materialization)
- ② 流水线执行 (Piplining) / 火山模型 (The Volcano Model)

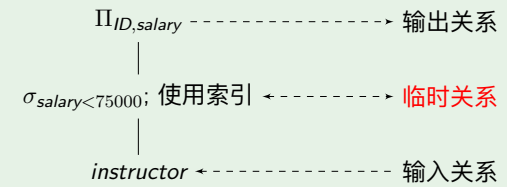
算子之间如何传递数据?

6.3.1 物化执行

物化执行 (Materialization)

- 自底向上执行查询计划中的算子
- 每个中间算子的执行结果写入临时关系文件，作为其后续算子的输入

例 (物化执行)



物化执行的缺点

缺点 1: 物化临时关系增加了查询执行的代价

- 执行完一个算子后，必须将临时关系写入文件（除非临时关系非常小）
- 执行后续算子时，需读入临时关系文件

缺点 2: 用户获得第一条查询结果的时间延迟大

6.3.2 流水线执行

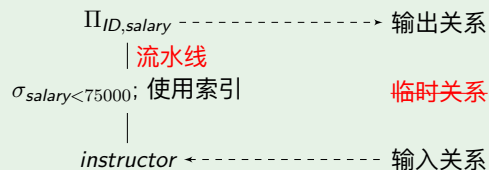
流水线执行 (Piplining)

将查询计划中若干算子组成流水线 (Pipeline)，其中一个算子的结果直接传给流水线中下一个算子

- 避免产生临时关系，消除了读写临时关系文件的 I/O 开销
- 用户能够尽早得到查询结果

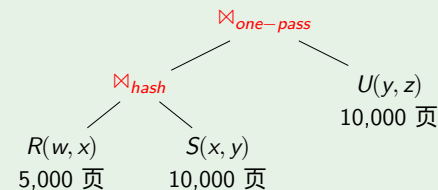
几乎所有 DBMS 都使用流水线执行查询计划

例 (流水线执行)

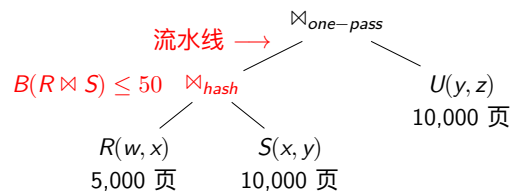


流水线查询执行

例 (流水线查询执行)

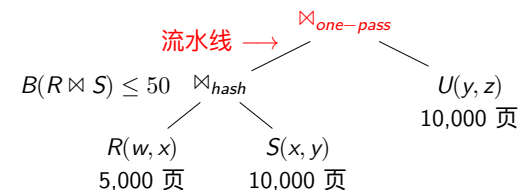


- 缓冲池中有 $M = 101$ 个可用页
- R, S, U 上均无索引且未按连接属性排序
- 设 $B(R \bowtie S) \leq 50$



使用哈希连接算法执行 $R \bowtie S$

- 哈希分桶阶段使用 101 页内存
输入缓冲 ☐ 1 页
100 个桶 ☐ 100 页
- 逐桶连接阶段使用 51 页内存 (不计输出缓冲)
 S 的输入缓冲 ☐ 1 页
 R 的桶 ☐ 50 页
输出缓冲 ☐ 50 页
- I/O 次数 = $3B(R) + 3B(S) = 45000$
- 因为 $B(R \bowtie S) \leq 50$, 所以 $R \bowtie S$ 的结果可以保留在输出缓冲区中, 以流水线的形式传递给下一个操作 $\bowtie_{one-pass}$



使用一趟连接算法执行 $R \bowtie S$ 的结果与 U 的连接

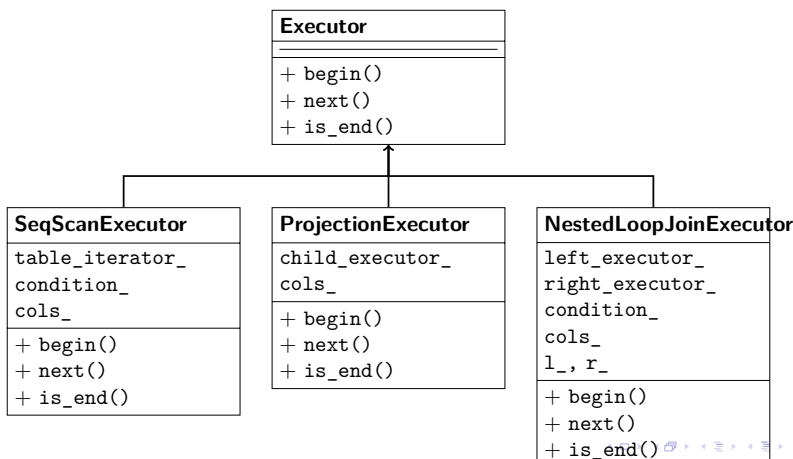
- 一趟连接算法使用 $B(R \bowtie S) + 1$ 页内存 (不计输出缓冲)
 U 的输入缓冲 ☐ 1 页
 $R \bowtie S$ 的结果 ☐ $B(R \bowtie S)$ 页 (已在缓冲池中)
输出缓冲 ☐ $100 - B(R \bowtie S)$ 页
- I/O 次数 = $B(U) = 10000$ ($R \bowtie S$ 的结果已在内存中, 无需 I/O)

6.3.3 火山模型/迭代器模型

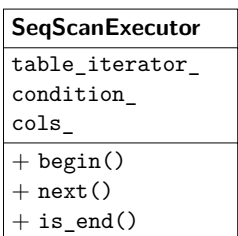
使用迭代器 (Iterator) 实现流水线中每个算子

- **begin()**: 定位到算子的第一个结果
- **next()**: 定位到算子的下一条结果
- **is_end()**: 判断是否到达算子结果的末尾
- 迭代器需要记录自身执行状态

算子的迭代器实现 利用面向对象编程的多态机制



顺序扫描 SeqScan 算子的迭代器实现



- 表记录迭代器 `table_iterator_`
- 选择条件 `condition_`
- 投影属性 `cols_`

```
Tuple* SeqScanExecutor::begin() {
    table_iterator_ -> begin();
    return next();
}

Tuple* SeqScanExecutor::next() {
    while (!table_iterator_ -> is_end()) {
        Tuple* r = table_iterator_ -> next();
        if (eval(r, condition_))
            return proj(r, cols_);
    }
}

bool SeqScanExecutor::is_end() {
    return table_iterator_ -> is_end();
}
```

投影算子的迭代器实现

SeqScanExecutor
table_iterator_ condition_ cols_
+ begin() + next() + is_end()

- 输入算子执行器child_iterator_
- 投影属性cols_

```
Tuple* ProjectionExecutor::begin() {
    Tuple* r = child_iterator_->begin();
    return proj(r, cols_);
}

Tuple* ProjectionExecutor::next() {
    if (!child_iterator_->is_end()) {
        Tuple* r = child_iterator_->next();
        return proj(r, cols_);
    }
}

bool ProjectionExecutor::is_end() {
    return child_iterator_->is_end();
}
```

嵌套循环连接算子的迭代器实现

NestedLoopJoinExecutor
left_executor_ right_executor_ condition_ cols_ l_, r_
+ begin() + next() + is_end()

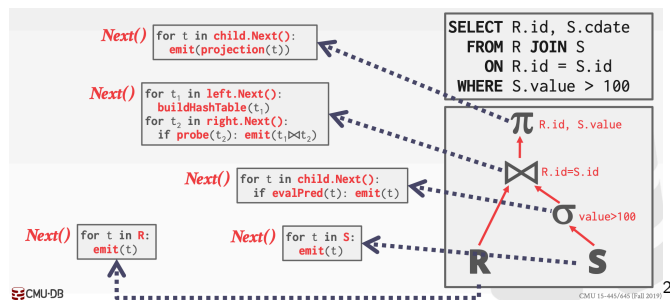
```
Tuple* ProjectionExecutor::begin() {
    l = left_iterator_->begin();
    r = right_iterator_->begin();
    if (eval(l, r, conditon_))
        return proj(join(l, r), cols_);
}

Tuple* ProjectionExecutor::next() {
    while (!left_iterator_->is_end()) {
        while (!right_iterator_->is_end()) {
            r = right_iterator_->next();
            if (eval(l, r, conditon_))
                return proj(join(l, r), cols_);
        }
        l = left_iterator_->next();
    }
}

bool ProjectionExecutor::is_end() {
    return left_iterator_->is_end();
}
```

火山模型

- 查询执行器调用查询计划最顶层算子迭代器的 next() 函数，获得下一条查询结果
- 一个算子为了获得输入，会调用其输入算子迭代器的 next() 函数
- 计算查询结果所需的数据不断从下向上收集，状如“火山喷发”



火山模型的不足

- 数据以元组为单位进行处理，不利于高速缓存发挥作用
- 为获得一条查询结果，需要多次调用查询计划中算子的 next() 函数
- next() 为虚函数，调用开销大
- 实验发现，open()、next() 和 close() 的运算逻辑（即真正的查询执行过程）所花费的时间只占查询执行总时间的 10%

²来源: Andy Pevlo, CMU 15-445/645

6.3.4 查询执行的优化

查询执行的优化

利用计算机系统的新特性来提高查询计划的执行效率

- ① 编译执行 (Compiled Execution)
- ② 向量化执行 (Vectorized Execution)

当代 CPU 的特性

超标量流水线 (Superscalar Pipeline) 与乱序执行 (Out-of-Order Execution)

- 超标量流水线 (Superscalar Pipeline)
- 乱序执行 (Out-of-Order Execution)
- 分支预测 (Branch Prediction)
- 单指令多数据流 (Single Instruction Multiple Data, SIMD)

- CPU 指令执行可以分为多个阶段，如取址、译码、取数、运算等
- **流水线**：一套指令控制单元可以同时执行多条指令，但每条指令所处的执行阶段不同，例如上一条指令执行到了取数阶段，下一条指令执行到了译码阶段
- **超标量**：一个 CPU 核中有多套控制单元，可以同时并行执行多个流水线
- **乱序执行**：CPU 维护了一个可乱序执行的指令窗口，窗口中无数据依赖的指令可以被取来并行执行

分支预测 (Branch Prediction)

跳转指令

- 程序分支越少，流水线效率越高，但程序分支不可避免
- 当执行一个跳转指令时，在得到跳转的目的地址之前，不知道该从哪里取下一条指令，流水线只能空闲等待

分支预测

- CPU 引入了一组寄存器，专门用来记录最近几次某个地址的跳转指令的目的地址
- 当再次执行到这个跳转指令时，就直接从上一次保存的目的地址取出指令，放入流水线
- 待获取到真正的目的地址时，如果预测的指令取错了，则抛弃当前流水线中的指令，取真正的指令进行执行

单指令多数据流 (Single Instruction Multiple Data, SIMD)

- 计算密集型程序通常会对大量不同的数据执行相同的运算
- SIMD 引入了一组大容量寄存器，每个寄存器包含 8×32 位，可同时存储 8 个 32 位数据
- CPU 新增了处理这种 8×32 位寄存器的 SIMD 指令，可以在一个指令周期内完成对寄存器内 8 个数据的相同运算

编译执行 (Compiled Execution)

拉取式 (Pull-based) 查询执行模型

- 火山模型采用了拉取式查询执行模型
- 大量虚函数调用导致性能损失

推送式 (Push-based) 查询执行模型

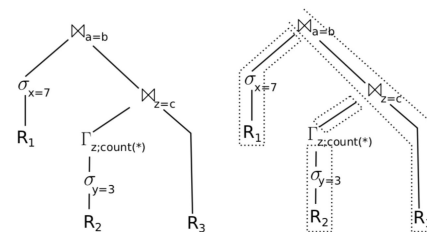
- 编译执行采用了推送式执行模型
- 与拉取式模型相反，推送式模型自低向上执行，执行逻辑由最底层算子开始，处理完一条元组之后，将元组传给上层算子继续进行处理

编译执行 (Compiled Execution)

- 编译执行将查询计划划分为多个流水线
- 在每个流水线内，数据可以一直留在寄存器中

例 (编译执行的流水线)

```
SELECT * FROM R1, R3,  
  (SELECT R2.z, COUNT(*) FROM R2 WHERE R2.y = 3 GROUP BY R2.z) AS R2  
WHERE R1.x = 7 AND R1.a = R3.b AND R2.z = R3.c;
```



编译执行 (Compiled Execution)

前面的查询计划对应的编译执行的伪代码³

- 每个流水线对应一个 for 循环
- 一次 for 循环处理一条元组，元组在一次 for 循环内不离开寄存器
- 编译执行的伪代码以数据为中心，消除了火山模型中大量的虚函数调用

```
initialize memory of  $\mathbb{M}_{a=b}$ ,  $\mathbb{M}_{c=z}$ , and  $\Gamma_z$ 
for each tuple  $t$  in  $R_1$ 
  if  $t.x = 7$ 
    materialize  $t$  in hash table of  $\mathbb{M}_{a=b}$ 
for each tuple  $t$  in  $R_2$ 
  if  $t.y = 3$ 
    aggregate  $t$  in hash table of  $\Gamma_z$ 
for each tuple  $t$  in  $\Gamma_z$ 
  materialize  $t$  in hash table of  $\mathbb{M}_{z=c}$ 
for each tuple  $t_3$  in  $R_3$ 
  for each match  $t_2$  in  $\mathbb{M}_{z=c}[t_3.c]$ 
    for each match  $t_1$  in  $\mathbb{M}_{a=b}[t_3.b]$ 
      output  $t_1 \circ t_2 \circ t_3$ 
```

³<https://zhuanlan.zhihu.com/p/63996040>

向量化执行 (Vectorized Execution)

- 向量化执行仍然采用拉取式查询执行模型
- 与火山模型的区别在于，迭代器 `next()` 函数的返回结果是一批数据（如 1000 行），而不是 1 条元组

向量化执行的优点

- 减少了火山模型中虚函数调用的次数
- 以块为单位处理数据，提高了缓存命中率
- 多行并发处理，符合当代 CPU 乱序执行与并发执行的特性
- 同时处理多行数据，使 SIMD 有了用武之地

总结

① 6.1 查询处理的过程

② 6.2 物理查询算子的实现算法

- 6.2.1 排序算子
- 6.2.2 选择算子
- 6.2.3 投影算子
- 6.2.4 去重算子
- 6.2.5 聚集算子
- 6.2.6 集合算子
- 6.2.7 连接算子

③ 6.3 查询执行模型

- 6.3.1 物化执行
- 6.3.2 流水线执行
- 6.3.3 火山模型/迭代器模型
- 6.3.4 查询执行的优化

习题

- 1 Describe the *one-pass aggregation algorithm* and analyze its I/O cost and memory requirement
- 2 Describe the *hash-based aggregation algorithm* and analyze its I/O cost and memory requirement
- 3 Describe the *sort-based aggregation algorithm* and analyze its I/O cost and memory requirement
- 4 Write pseudocode for an iterator that implements a join algorithm (*one-pass join*, *block-based nested loop join*, *sort-merge join*, *hash join*, *index-based join*)

致谢

2026 版以前

- 感谢徐仕涵同学 (2022113588) 指出并修正课件中的错误。